Hardware-isolated AI agents on EKS with Nitro Sandboxes: a hands-on guide, with a gVisor comparison

Author: Igor Gorovyy, DevOps Engineer Lead & Senior Solutions Architect, AWS Community Builder

You let an AI agent write code. That code came out of a model that reads the internet, and you cannot trust it. So the execution environment needs a real isolation boundary. On November 9, 2026, AWS shipped a new one: Nitro Sandboxes, lightweight VMs managed by the Nitro Hypervisor. They work for EKS pods, ECS tasks and standalone EC2 VMs, at no extra charge.

Here is how it worked in my practice: a cluster where Nitro Sandboxes, gVisor and plain runc run side by side on identical instances, the “agent runs code in an isolated pod” pattern wired end to end, and measurements of what the isolation costs in start time and density.

Three isolation levels, on the same instance

Run one `m8i.xlarge` three ways and the difference is visible with `uname -r` on a pod:

Runtime	Boundary	Kernel the pod runs
runc (the default)	namespaces, cgroups	the host kernel (AL2023)
gVisor via `runsc`	user-space syscall interception	a synthetic kernel, the Sentry
Nitro Sandbox	a hardware VM under the Nitro Hypervisor	its own guest kernel

On gVisor, briefly: it reimplements a Linux kernel in a user-space process. It sounds like a trick, and it works well on any instance type.

Nitro Sandbox is organized differently. Each pod becomes a micro-VM the way a Lambda function is one: the pod container runs inside a VM with its own kernel, started on the same EC2 instance you already rent. Two consequences follow:

1. Secrets can be released only to sandboxes that prove their integrity. The sandbox’s measurements live in PCR registers, and a KMS key policy with `kms:RecipientAttestation:PCR` conditions releases decryption keys only on a match. But here is the problem, straight from the docs: without that key policy (or another external verifier) the measurement is recorded and never checked. For regulated workloads, and for anything where an agent holds credentials, that is the critical detail.
2. The isolation boundary is hardware. The same one AWS puts between your instances and between a Lambda workload and its host.

The old road to “a real VM inside a pod” was Kata Containers on nested virtualization. It still works, and it now comes with an asterisk. Nested virtualization exists only on 8th-gen Intel instances

(c8i/m8i/r8i), it is not available on Graviton, and there is no attestation. Nitro Sandboxes lift all three.

Bringing up the cluster

EKS Auto Mode with Kubernetes 1.36 or newer, and enabling Nitro Sandboxes for a pod looks like this:

```
spec:
  runtimeClassName: isolated-sandbox
```

Everything else in the pod stays native: Pod Identity, VPC CNI, EBS/EFS/S3 volumes, CloudWatch all work as usual.

My cluster (full Terraform here), one Auto Mode cluster with a NodePool pinned to `m8i.xlarge`, plus gVisor on a managed node group using the same instance type:

```
# Auto Mode NodePool for Nitro Sandbox pods
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: sandbox-nitro
spec:
  template:
    metadata:
      labels: { sandbox-runtime: nitro }
    spec:
      taints:
        - { key: sandbox, value: nitro, effect: NoSchedule }
      requirements:
        - key: eks.amazonaws.com/instance-family
          operator: In
          values: ["m8i"]
```

```yaml
# One of the four smoke-test pods
apiVersion: v1
kind: Pod
metadata:
  name: nitro-codeexec
  annotations:
    sandbox.amazonaws.com/network-isolated: "true"   # no network device at all
spec:
  runtimeClassName: isolated-sandbox
  nodeSelector: { sandbox-runtime: nitro }
  tolerations:
    - { key: sandbox, operator: Equal, value: nitro, effect: NoSchedule }
```

Two spots you will run into:

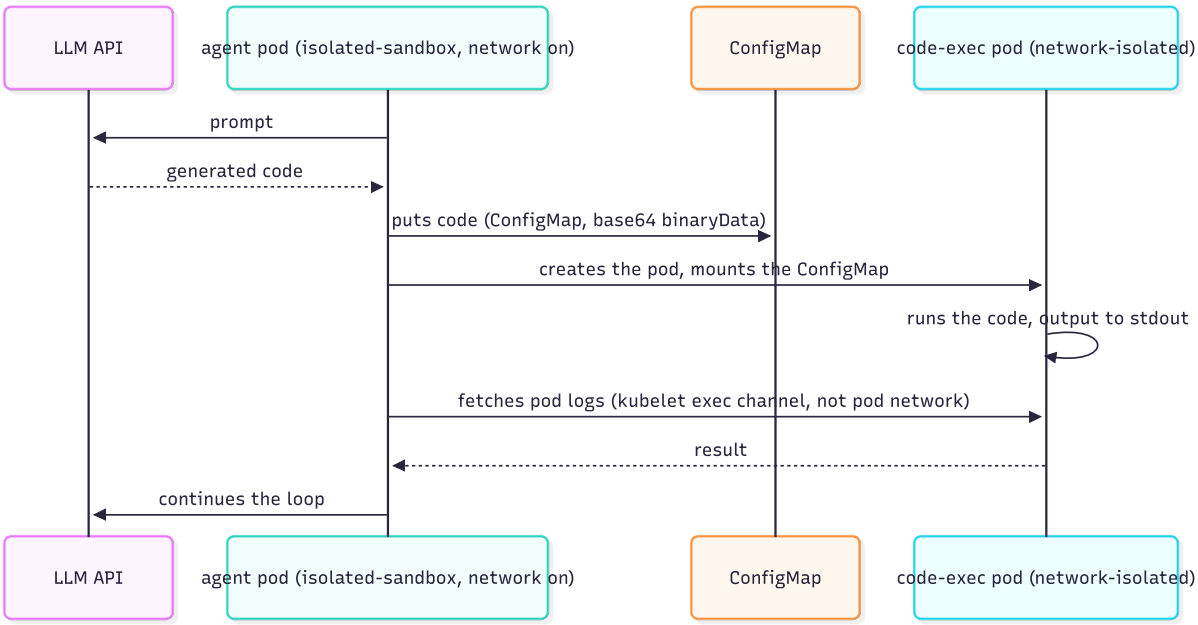
- Pods without the taint toleration and nodeSelector land on the standard runtime. The isolation is opt-in, per pod, not per cluster.
- The `network-isolated` annotation removes the pod's network device entirely: no DNS, no IMDS, no Pod Identity credentials. Side effect: HTTP and TCP probes stop working, so such pods take exec probes only. `kubectl port-forward` does not reach a network-isolated pod at all; logs, exec and stats travel over a vsock channel that does not depend on the pod network.

The full bring-up on a default VPC took about 20 minutes with `terraform apply` (31 resources: private subnets, a NAT gateway, the cluster, admin add-ons, two node groups). The interesting parts live in three files: the NodePool above, a `RuntimeClass` for gVisor, and a NodeConfig that teaches containerd about `runsc` and installs gVisor on node boot.

After the cluster came up, four probe pods confirmed the diagram from the table: runc printed the host kernel (`6.18.56` , shared with its node), the Nitro Sandbox pod printed a different guest kernel (`6.18.54`) and exactly the vCPU count it requested, the network-isolated pod saw only `lo` and a dead DNS (`wget: bad address` , then `NETWORK: blocked`), and the gVisor pod printed the Sentry kernel (`4.19.0-gvisor`). Full outputs live in the repo, `evidence/runtime-evidence.txt` .

The pattern: agent with network, code without it

The pattern has two roles:



The agent pod keeps its network: it talks to the model API and to the Kubernetes API. The code runs in a separate pod with no network device, no Pod Identity and no service account token. The two never share a socket, only API objects.

Three implementation spots where I spent the most time:

emptyDir does not work for this, in two different ways. It is pod-scoped, so two pods cannot share one. And putting both containers into one pod does not help either: the `network-isolated` annotation is pod-level, and the agent would lose its network too. This is why the exchange goes through the API server.

The code travels as base64 in `binaryData`. Kubernetes mounts a `binaryData` entry as the decoded bytes, and in shell that means zero JSON escaping. The agent script has no kubectl and no jq inside the sandbox VM; raw REST calls via curl over the service-account token and CA that kubelet already mounts work fine. alpine plus `apk add curl` is the whole dependency list.

The code-exec pod mounts no service account at all. Set `automountServiceAccountToken: false` on it. Then the generated code cannot call the Kubernetes API, create pods, or read secrets, even though it already has no network. The RBAC around the agent is namespace-scoped: create and delete pods and ConfigMaps in its own namespace, get pod logs, nothing else.

The full agent loop fits into one ConfigMap script and one pod spec; the manifest lives here. In the demo, the generated code checks its own isolation. It ran to completion in the network-isolated pod and printed:

```
 kernel: 6.18.54
interfaces: dummy0 lo
wget: bad address 'aws.amazon.com'
NETWORK: blocked — isolation works
SA token: absent — expected
uid: 0
```

Three claims from this section held on a live cluster: the ConfigMap mounts inside the sandbox VM, kubelet exec and log access work with no network device, and the service-account token is absent with `automountServiceAccountToken: false`.

What isolation costs

Method. For cold start: N pods of each runtime, time from `kubectl apply` to Ready, measured per pod from `creationTimestamp` to the Ready condition, plus wall time. Run twice, the first run includes node bootstrap. For density: keep applying 1-vCPU `sleep` pods until Ready stops growing. One methodology note found in practice: the scheduler packs pods by CPU *request*, so to reach the sandbox slot budget at all I set the request to 250m with a 1-vCPU limit. With a full 1-vCPU request the slot budget never gets to bind (observed live: one pod per busy node where three smoke pods already held the vCPUs).

Metric	runc	Nitro Sandbox	gVisor
Cold start on a warm node, per pod	1-5 s (9 runs)	3-4 s best, 37 s p90	n/a (censored, see below)
Density: pods per m8i.xlarge	3-4	15 Ready out of a 20-slot cap	2
Node cost per pod-hour	~$0.07	~$0.02	~$0.12
Cold start including node bootstrap	215 s wall for 10 pods (4 new nodes)	same window + sandbox boot	n/a

What to expect in your own run:

- Density. The docs are explicit here. Two caps apply per node, both sized at 5 times the node's vCPU count: the total vCPUs allocated across all sandboxes, and the number of sandboxes (the count cap is halved on CPU-overcommit instance types). A 4-vCPU `m8i.xlarge` therefore fits up to 20 single-vCPU sandboxes. Each sandbox pod also carries a fixed memory overhead the scheduler adds on top of the pod request: 256 MiB for `isolated-sandbox`, 512 MiB for `confidential-sandbox`. And on SMT instance types (all Intel, most AMD), core scheduling keeps two different sandboxes off the sibling threads of one core, so compute available to sandboxed pods is roughly half the advertised vCPUs; Graviton, one thread per core, is unaffected. In my run memory bit first: 15 pods of 512 Mi + 256 Mi overhead fit into 14.4 GiB allocatable, and 5 of 20

stayed Pending. gVisor has none of these caps, so the crossover is where density beats boundary strength.

- Cost. Auto Mode adds a management fee on top of EC2 that managed node groups do not have: /bin/zsh.02831 per hour for an m8i.xlarge node in eu-west-1 (checked against the public price list; ~12% of the EC2 rate). With it included, the packed-node numbers in the table stay the same rounding. NAT for the private subnets is about $0.045 per hour, and it bills whether or not your cluster has traffic, so tear the demo down after the run.

Cold start is two numbers, not one

The first benchmark run taught more than the CLI output shows. A pod's time-to-Ready decomposes into two independent pieces, and they measure different risks.

Sandbox startup on a node with free slots. A warm node brings a pod to Ready in 3-37 s (runc: 1-5 s). This is the honest cost of the isolation itself: image pull, VM boot, guest kernel init.

Node provisioning. A pod that does not fit lands in the scheduler's bin-packing decision. When it nominates a new nodeclaim, EC2 has to launch a node first, and that is 1-3 minutes of AMI boot plus kubelet join before any of your pods moves. My first wave measured 215 s wall time; the second, warm run of the same spec measured 5 s. The difference is node bootstrap, not the sandbox.

And it can get much worse for a reason unrelated to sandboxes. My account hit **VcpuLimitExceeded** on the on-demand instance family: the default limit for a fresh account is 32 vCPUs per family. Seven m8i.xlarge nodes at 4 vCPUs each plus the system node filled the bucket, and every new nodeclaim retried with `InsufficientCapacityError` while pods sat in Pending for about half an hour until I freed capacity. If you repeat this: before density experiments, raise the family vCPU limit or delete the nodeclaims you no longer need.

A practical consequence for benchmark methodology: measure the sandbox cost on warm nodes, and measure "from nothing to running" as a separate cold path that includes node bootstrap. Do not blend the two; they scale differently and break differently.

Where it does not fit

- No GPU passthrough yet. Local inference inside a Nitro Sandbox is not possible today (the sandbox runs on Firecracker, which does not pass devices through). An agent with a local model falls back to the API, or to gVisor, which inherits host GPUs.
- No resize, no snapshot. CPU and memory are allocated at pod boot: in-place resize is not supported, changing them means deleting and recreating the pod with new requests. Snapshot and restore and live migration are absent too, so idle agents keep paying for their resources.
- The agent's own privileges are the real risk. Generated code is well fenced in this pattern: no network, no token, no API. But the agent pod holds a create-pods permission in its namespace, and admission today does not gate its pod specs. For production this needs a Pod Security Standards policy on the namespace and an admission gate on `runtimeClassName` and hostPath,

or moving pod creation out of the model-facing pod. The demo does not close this gap by itself, and hiding it in an article would be dishonest.

- Public cluster endpoint. This guide's cluster keeps `endpoint_public_access` on without a CIDR allowlist for the sake of a short demo. In anything long-lived, restrict it (`endpoint_public_access_cidrs`) or drop it: private access is already on.
- The vsock channel is a known surface. The docs say it themselves: the vsock device used for shim-to-agent communication is accessible to processes inside the guest, and any parent process with access to the Firecracker vsock socket can connect guest-bound ports, including on network-isolated pods. Not a classic network bypass, but the "no network device" line deserves this asterisk in a security review.

Takeaways

For agents that execute model-written code on EKS today:

1. Nitro Sandboxes: the cheapest road to a hardware isolation boundary per pod, and the only one with attestation. Enablement is one line, and the pod stays native.
2. gVisor still wins where density matters more than boundary strength, and where GPUs matter.
3. Nested virtualization for Kata is the legacy path now. If you have it running, fine. For new work: Nitro Sandboxes replace it on the same instance class you may already be running.

The pattern above is small enough for one person to own, and strict enough to survive a security review. After the two admission gaps from the last section are closed.

All Terraform, manifests, the agent pod setup and the benchmark script are in github.com/igorgorovoy/eks-sandboxes. The numbers in the tables come from the runs recorded in `benchmark-results.csv`.

Appendix: raw measurement data

Everything below reads straight from `benchmark-results.csv` in the repo; times are per pod from `kubectl apply` to Ready.

Cold start, warm node (second run), per pod, seconds:

runc	nitro isolated	nitro network-isolated	gVisor
2, 1, 1, 1, 1, 1, 0, 0, 0, 1	4, 36, 36, 36, 37 (+1 pod at 1742: it waited for a new node)	37, 38, 38, 38, 38, 37, 37 (+3 pending past the window)	0, 0 (two pods; Ready flipped within the same second) + 8 pending

Wall time for the batches: runc 5 s, nitro 1802 s (dominated by node-wait), nitro-net 1808 s, gVisor 902 s.

Cold start including node bootstrap (first run), runc, per pod, seconds: 37, 37, 37, 209, 209, 209, 209, 1, 1, 1. Wall 215 s across 4 nodes: three waves of pods, one per new node plus one wave on the node

that already existed.

Density, pods Ready per node (60 pods offered, 250m request / 1 vCPU limit):

Runtime	Ready per node	Total
runc	3, 3, 3, 3 (four nodes) + 1 (on the busy nitro node)	13/60
Nitro Sandbox	15, 5 Pending	15/20 offered
gVisor	2	2/60

A negative number in the CSV means the pod was never Ready inside the measurement window; those rows map to the Pending counts above.

Sources

- Sandboxing on Amazon EKS: key features, attestation, encrypted images
- Nitro Sandboxes considerations, EKS: density caps, SMT core scheduling, resize and device limitations
- Network isolation: exec probes, vsock channel behavior
- Autoscaling with Nitro Sandboxes: slot overhead (256/512 MiB), provisioning
- Setup paths on EKS: Auto Mode vs node groups, K8s >= 1.36
- Amazon EC2 Nitro Sandboxes: instance support (Intel/AMD/Graviton), when to use what
- gVisor documentation
- The repo with all files from this guide - try it